Budget IoT Devices and their Network Vulnerabilities

Julio Arellano

University of Arizona

CYBV 498: Senior Capstone – Engineering

Casey Cota

April 21, 2026

Abstract

Budget Internet of Things (IoT) devices have expanded the consumer attack surface through manufactured cost-cutting strategies that prioritize low prices over security. This research examines the network security of low-cost IoT devices through the development of a custom Python-based security scanner using a three-stage methodology: ARP discovery, targeted port scanning for IoT associated services, and optional packet capture. Testing on a small local subnet revealed six vulnerable devices with exploitable configurations. These included a potentially unauthenticated RTSP camera stream, open HTTP interfaces on smart plugs susceptible to default credential attacks, and a Tuya-based device vulnerable to permanent takeover via the CloudCutter exploit. The research findings show that cost-cutting decisions directly create exploitable vulnerabilities. Attack scenarios show that an attacker with temporary local network access could gain access to private video feeds, control appliances, compromise network gateways, or completely hijack devices. The paper concludes with recommendations for both consumers (network segmentation, credential changes) and manufacturers (unique device credentials, consistent firmware updates). If these dynamics between manufacturers and cost-cutting strategies stay the same, budget IoT devices will remain an ongoing threat to home network security for the foreseeable future.

Table of Contents

The Rise of IoT 5
Literature Review 6
Embedded Software and Connectivity Vulnerabilities 7
Real World Exploitation of IoT Vulnerabilities 8
Gaps in Current IoT Security Research 9
Synthesis 9
Recent IoT-Based Attacks 10
Mirai Botnet 10
IP Cameras 11
Depreciated Components and Their Risks 12
Market Pressure to Maximize Profits 12
Security for the Average Consumer 14
The Invisible Burden of Updates 15
The Consumer Frustration Feedback Loop 16
An Analysis of Budget IoT Devices 18
Development Environment and Language Selection 18
Library Selection and Justification 19
Risk Scoring Methodology 20
Device Classification 20
Script Prototyping 21
Psuedocode for the Vulnerability Scanner 22
Script Compilation and Execution 23
Test Device Selection 24
Network Discovery Result 25
Risk Score Distribution 27
Threat Analysis 27
Scenario 1: Unauthorized Camera Surveillance 28
Scenario 2: Smart Plug Control and Network Pivot 28
Scenario 3: Gateway Compromise 28
Scenario 4: Tuya CloudCutter Permanent Device Takeover 29

Comparison to Consumer Expectation 29
Conclusion 30
References 32

The Rise of IoT

The Internet of Things (IoT) was sold to consumers as a way to seamlessly unite the digital and physical world. The idea of a "smart home" that had devices that anticipate and respond to human needs has been a staple of science fiction since the twentieth century. Today, that fiction has become reality. Smart speakers respond to voice commands like attentive butlers. Algorithm-driven thermostats learn consumer temperature preferences and adjust automatically. Doorbell security cameras stream live video to a user's phone in an instant, allowing homeowners to monitor their property from anywhere on the globe. These technologies have become so integrated into the daily lives of the current generation, that the thought of a time when such conveniences did not exist is difficult to imagine.

The first IoT device began its life as a student project in the 1980s at Carnegie Mellon University. A simple modification to a Coca-Cola machine that tracked and reported soda stock levels became the grandfather of millions of devices currently on the market (Teicher, 2025). Over the following decades, the consumer desire for convenience increased. In 2000, LG launched the first smart refrigerator that had internet access and multimedia capabilities. In the following decades, the tech market was inundated with IoT devices that dominated the industry. 2014 saw the release of Amazon's Alexa and in 2015 iRobot's Roomba vacuums began adopting IoT design philosophies. In fact, by 2024 the average U.S. household had an average of 21 connected IoT devices (Blinder, 2024).

This explosive growth has been driven in part by the plunging prices caused by a competitive market. Products that once required a significant monetary barrier to entry can now be purchased for less than a meal. Smarts plugs litter Amazon for under ten dollars. Security cameras can be purchased for less than the price of a video game. Artificial intelligence voice

assistants have become so simple to implement that they are often tacked onto products that do not benefit from their inclusion. The accessibility and standardization of “smart” technologies have made IoT devices accessible to nearly every possible consumer, but this has come at a cost that is not reflected by price tags.

Achieving such low prices requires strategic compromises in device design, component selection, and software maintenance. Manufacturers under pressure to capture market share may choose to use deprecated code libraries rather than developing new ones, integrate outdated hardware components available at discount, or cease firmware updates shortly after product release to maximize profit. In their security analysis of low-budget smart home appliances, Murat et al. (2024) documented numerous cases where these cost-cutting measures translated directly into exploitable vulnerabilities including unencrypted network traffic, hard-coded credentials, and known vulnerabilities in reused software components.

This paper aims to examine the network vulnerabilities introduced by budget IoT devices through direct analysis of their behavior on local networks. By developing a script to intercept and dissect data transmitted through these devices, this research aims to document exactly what information is exposed, and how that exposure could potentially be leveraged by bad actors.

Literature Review

By performing a literature review, the intent is to provide empirical documentation of exposed data, improve upon network vulnerability methodologies, discuss gaps in current IoT security practices, and determine whether cost-cutting strategies employed by manufacturers have led to direct and measurable consumer risk.

Embedded Software and Connectivity Vulnerabilities

Security vulnerabilities in the budget IoT market have been consistently studied across both small and large-scale firmware analysis. Research shows that vulnerabilities stem from cost-cutting measures such as reused software components, outdated encryption protocols, and insufficient firmware updates. Murat et al. (2024) conducted a direct security analysis of budget smart home appliances, focusing specifically on embedded software and network connectivity. Their research revealed exposed debugging interfaces, outdated TLS encryption protocols, and insecure firmware update mechanisms.

During their analysis of a Tuya-based smart plug, Murat et al. (2024) successfully extracted the device's firmware and reverse engineered it. The research demonstrated that the firmware could be modified to transmit sensitive data to an attacker-controlled server. This finding highlights the vulnerabilities introduced by insecure firmware update mechanisms and demonstrates how bad actors could gain permanent access to home networks through infected IoT devices.

While Murat et al. (2024) focused on individual device analysis, Chen et al. (2024) focused on identifying vulnerabilities at scale. Their large-scale study analyzed more than 6,900 firmware images and identified 2,700 vulnerabilities affecting 349 open-source libraries. Their findings show that manufacturers updated a vulnerable software library only 32.7% of the time. Their research also found that manufacturers took an average of 1.34 years to patch vulnerable libraries, leaving devices exposed for extended periods of time.

Together, these studies show that vulnerabilities in budget IoT devices are not isolated incidents and are in fact, a systemic issue persistent in the IoT manufacturing ecosystem. Murat

et al. (2024) demonstrated how attackers could exploit individual devices through insecure firmware configurations, while Chen et al. (2024) showed that those vulnerabilities are a widespread issue across thousands of devices. The combination of insecure firmware and delayed updates increases the likelihood that vulnerable devices remain exposed long enough to be discovered and exploited by bad actors.

Real World Exploitation of IoT Vulnerabilities

The vulnerabilities identified in IoT firmware have already been exploited in a live setting. Antonakais et al. (2017) conducted a landmark retrospective analysis of the Mirai botnet. Their research combined multiple methodologies, such as Internet-wide scanning and Telnet honeypots that allowed them to track Mirai's growth.

Mirai spread by scanning random IPv4 addresses and attempting to log into devices using default credentials over Telnet ports 23 and 2323. Once access was gained, the malware uploaded device information to a separate server and infected the device. During their monitoring period, Antonakakis et al. (2017) documented over 15,000 attacks targeting game servers, telecom providers, anti-DDoS services and political websites. The quality of research has made their article a foundational resource for countless research papers on IoT vulnerabilities and malware.

The success of Mirai can be linked to the types of vulnerabilities identified by Murat et al. (2024) and Chen et al. (2024). Default credentials, outdated software libraries, and lack of firmware updates created the perfect conditions for a botnet like Mirai to propagate. Antonakakis et al. (2017) illustrated how the vulnerabilities the previous research reported on result in large-

scale consequences for the Internet. The widespread popularization of budget IoT devices, combined with poor security practices create an environment where botnets can easily form.

Gaps in Current IoT Security Research

Although existing research has identified vulnerabilities in budget IoT devices and documented real-world examples of exploitation, there remains a gap in accessible methods for identifying vulnerable devices on local networks. Murat et al. (2024) relied on laboratory-based firmware extraction and reverse engineering, which requires specialized hardware, software and technical expertise. Chen et al. (2024) performed large-scale analysis of firmware images but focused more on vulnerabilities within software libraries rather than smaller home networks. Antonakakis et al. (2017) conducted Internet-wide attack measurement and botnet tracking, rather than individual network security.

While these studies provide valuable insight into IoT vulnerabilities, they do not provide practical tools for everyday users or researchers to identify vulnerable devices within their own local networks. This limitation highlights the need for accessible methods for detecting insecure IoT devices in an everyday home environment.

Synthesis

The literature reviewed confirms that low-cost IoT devices frequently contain exploitable vulnerabilities due to insecure firmware design, outdated software libraries and poor updating practices. Murat et al. (2024) demonstrated these vulnerabilities through experimentation, Chen et al. (2024) showed that vulnerable software is present across thousands of devices and Antonakakis et al. (2017) document how those vulnerabilities can be exploited at scale through botnets such as Mirai.

The research presented in this paper builds upon these findings by developing a network sniffing script that identifies vulnerable IoT devices on local networks. While previous studies focused on large-scale Internet research, this research aims to provide an accessible method for identifying insecure IoT devices within small local network environments. By addressing this gap, this study contributes to improving practical IoT security assessment and increasing awareness of vulnerabilities introduced by budget IoT devices.

Recent IoT-Based Attacks

As the market for IoT devices continues to grow, so does the potential attack surface. This large-scale vulnerability has been the target of numerous cyber-attacks in recent years, demonstrating that compromises made by budget device manufacturers have real world consequences for consumers and internet infrastructure.

Mirai Botnet

The most notorious example of a recent IoT attack is the Mirai botnet that emerged in 2016. The attack forever changed how cybersecurity researchers understood the threats caused by vulnerabilities inherent in consumer grade IoT devices. Mirai scanned the internet for IoT devices protected by factory default username and password combinations. Once infected, these devices became a massive botnet capable of launching distributed denial of service (DDoS) attacks at an unprecedented scale. In October 2016, this army of infected devices launched a DDoS attack against Dyn, a DNS provider at the time, causing websites like Reddit, Spotify, and

Twitter (currently known as X) to become unreachable by devices (Antonakakis et al., 2017). Figure 1 shows a heatmap of areas affected by the Mirai attack in the United States.

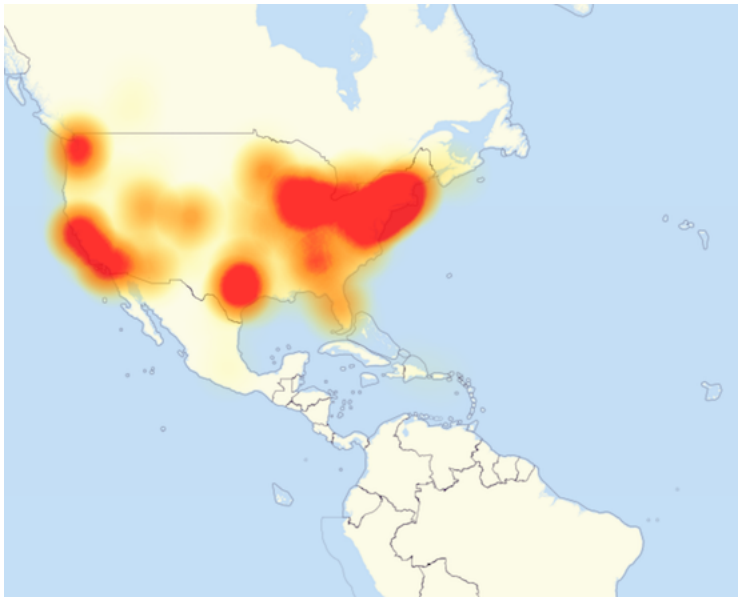

Figure 1 - Heatmap of areas affected by Mirai botnet attack on DYN servers.

What made Mirai so destructive was not just its technical sophistication, but its reach. The number of infected devices grew into the hundreds of thousands, many of which included IP cameras and home routers. According to Antonakakis et al. (2017), the Mirai botnet disproportionately infected regions such as Brazil, Columbia, and Vietnam. In black market circles, regions such as these were known to be low quality hosts for proxies and DDoS. One thing these countries have in common is that they are all developing countries. Consumers in these regions do not have as much disposable income, thus manufacturers must cut costs to provide more affordable IoT devices. The author of the Mirai botnet saw a vulnerability in an emerging attack surface and seized the opportunity, forever changing the landscape of network security.

IP Cameras

IP cameras have emerged as attractive targets when attacking IoT devices. In the 2010s, multiple news outlets reported on stories of hacked IP cameras. The LeMay family told CNN about a harrowing encounter that their 8-year-old daughter experienced when a malicious actor gained access to an unsecured Ring camera installed in her bedroom. The virtual intruder harassed, frightened, and encouraged the child to destroy their room (Wolfe & Ries, 2019). In 2012, an exploit was discovered that allowed unauthorized access to TRENDnet home security cameras. Users from websites such as 4chan and Reddit began to hack into the vulnerable devices and created websites that compiled live video feeds of homes to anyone with an internet connection. Though meant to protect and provide peace of mind to consumers, these devices can be exploited by bad actors to gain unauthorized access to private video feeds.

Deprecated Components and Their Risks

As IoT devices become a staple of the modern home, it is important to remember that over a decade's worth of devices are still connected to the internet. Many of these devices are running outdated firmware or are protected by manufacturer-set default credentials. A majority of these devices have reached end-of-life status, meaning that manufacturers no longer provide software and security updates. According to Chen et al. (2024), these deprecated libraries are still being used in modern IoT devices. Chen et al. (2024) found 11,342 vulnerabilities caused by 349 reused libraries. These libraries have been used in over 6,000 firmware images. Bad actors have had decades to reverse-engineer the libraries, and the lack of security updates for these devices traps these libraries in a perpetually vulnerable state for their entire lifetime.

Market Pressure to Maximize Profits

The rapid market expansion of IoT devices is a technological and economical phenomenon, driven by intense market pressure to maximize profit margines in a competitive sector. Manufacturers face a difficult problem: deliver feature-rich, smart functionality at a price point that is accessible to consumers, while still maintaining profitability. When these two objectives are considered, device security is one of the areas where cost cutting measures are applied. The result of which is a tier of devices designed to meet the bare minimum functionality required to justify the purchase, but not secure enough to withstand the evolving exploit landscape.

The pressure to maximize profits manifests itself in several engineering and business decisions. Firstly, hardware components are selected based on material cost minimization rather than their security properties. Low-end microcontrollers often lack hardware-accelerated encryption, secure boot mechanisms, or even sufficient memory to implement modern security protocols. Secondly, software development prioritizes speed over proper security best practices. Rather than writing new code, manufacturers frequently reuse deprecated open-source libraries with known vulnerabilities. As Chen et al. (2024) documented, they identified over 11,000 vulnerabilities across more than 6,000 firmware images. Lastly, post-sale support is not prioritized by budget manufacturers. Many budget devices receive firmware updates for only a few months after release, if at all, leaving them in a permanently exposed state while vulnerability identification can progress.

The economic logic is brutal, but clear. For a manufacturer selling millions of units at a sub-$20 price point, the cost of maintaining long-term security updates eats into the already thin profit margins. The manufacturers would have to invest in additional software development time and create infrastructure to push and deliver updates. Jaigirdar et al. (2016) found that systematic

vulnerability management in IoT systems requires dedicated resources that low-margin product lines simply cannot sustain. Because of this, devices are effectively abandoned shortly after their launch. Consumers, unaware of this lifecycle limitation, continue to use these devices for years, thus transforming their home network into a large attack surface.

These profit-driven vulnerabilities are not just hypotheticals. As mentioned previously, the Mirai botnet, which Antonakakis et al. (2017) analyzed in comprehensive detail, succeeded precisely because manufacturers had chosen to ship out devices with hard-coded or default credentials rather than investing in unique credentials for each device. Mirai's scanning mechanism exploited this predictable failure across hundreds of thousands of devices, demonstrating how individual manufacturer cost-cutting decisions compound into large-scale internet risk. In their behavioral analysis of IoT botnets, Rawat et al. (2025) confirmed that the majority of compromised devices continues to be those with unchanged default credentials or unpatched firmware, which are direct consequences of market-driven security neglect.

Security for the Average Consumer

Beyond purely economic pressures, budget IoT devices are often rendered insecure by a second, more subtle force. Budget manufacturers understand that their devices must be simple enough to be operable by the average consumer. Device security and usability exist in a constant struggle with one another. A perfectly secure device might require hardware tokens, certificate management, and regular manual updates, expectations that would alienate most users. Aware that their products must work "out of the box", manufacturers choose to sacrifice security in the name of accessibility.

Circling back once again to the Mirai botnet, we know that it was successful due to it abusing hard-coded generic credentials. The reason that manufacturers choose to not include unique security credentials is simple, user testing consistently demonstrates that friction during setup drives product returns and negative reviews. A consumer who needs to download a manual and figure out how to navigate a multi-step credential creation process is more likely to abandon setup entirely. Ronen et al. (2018) observed a similar phenomenon in their analysis of the Zigbee-based IoT attacks, noting that manufacturers of smart lighting and sensor devices routinely disabled encryption or used fixed network keys specifically to simplify device pairing. The consumer expects to be able to plug in the device and begin using it immediately. Adding cryptographic handshakes, certificate validation, or unique key exchanges add a layer of complexity that the average consumer is not expecting. The result is a category of devices that prioritize seamlessness over security because seamlessness sells and security does not.

The Invisible Burden of Updates

Automatic firmware updates represent another area where usability takes precedent over security. From a security perspective, devices should update automatically, silently, and frequently. From a usability perspective, automatic updates introduce risks that manufacturers and consumers fear. Rushed, unpolished updates can brick devices, change UI behavior in ways that confuse users, and consume valuable resources during operation. To put it simply, the average consumer does not want to think about their lightbulb's operating system or firmware version.

The fear of update-induced failures is not irrational. Real consequences of automatic updates are well-documented. In one case, a routine over-the-air update intended to add new features instead introduced a change in a proprietary document format that resulted in data loss

over 300,000 devices. The consequences for pushing out such a destructive update were devastating. Significant brand damage, customer impact, and team member terminations were observed (Siryi, 2025). Even when updates do not cause permanent damage, the perception of risk looms over manufacturers during decision making. A single high-profile update failure can destroy consumer trust in a brand far more effectively than a security vulnerability that remains invisible to the end user.

In their analysis of over-the-air updates, Siryi (2025) focuses on the fact that most critical decisions about update infrastructure must be made before the device first ships. Choosing hardware that lacks rollback capabilities, or neglecting secure boot and key management can result in limitations that cannot be resolved once devices are sold and deployed. For budget manufacturers operating on razor-thin margins, the upfront investment to build a robust, secure, and user-transparent update infrastructure is not realistic. These manufacturers face a choice: invest in a security infrastructure that may cost more than the entire device's development budget or ship the device as-is and hope those issues never become a problem.

This economic reality leads to the worst possible outcome from a security perspective. IoT devices that could receive security updates do not, because the manufacturer does not have the infrastructure in place to deliver them. Miettinen et al. (2017) observed that many IoT vendors produce devices that, "tend to lack mechanisms for firmware updates or patches that can help eliminate security vulnerabilities". The devices these manufacturers create become unpatchable by design and are left to degrade in a vulnerable state for their entire lifespan.

The Consumer Frustration Feedback Loop

Research consistently demonstrates that setup and connectivity issues are the primary reason that IoT devices are returned or abandoned. A study done by iQor (2018), found that more than one in three U.S adults experience issues setting up or operating IoT devices, with consumers spending an average of 2.5 hours between self-help and customer support to resolve their issues. 22 percent of these consumers dealing with connectivity issues chose to give up setup completely and return the product for a refund.

Even when devices remain in consumer's homes, many never function as intended. EE's research into U.K. smart home adoption revealed that one in three consumers lose interest in a new gadget if it cannot be connected within five minutes. Nearly the same number of consumers admit that Bluetooth pairing leads them to "push some buttons until something works" (Williams, 2025). The result is a graveyard of abandoned smart devices with 19 percent of consumers reporting that they have had at least one device disconnected for over three months and 13 percent with devices disconnected for over six (Williams, 2025).

This feedback loop of frustration reinforces manufacturers to prioritize simplicity over security. When consumers return devices because setup was too complicated, manufacturers receive a clear market signal to make the product easier to use instead of more secure. A device that requires a consumer to understand firmware versions, manually approve updates, or troubleshoot connectivity failures is a device that will be returned. The manufacturer that adds an extra validation step that will make the device more secure will lose sales to the manufacturer that chooses to omit it.

Consumers think that they want security, yet secure practices are punished in the market. They want their devices to be safe from bad actors but refuse to engage with any security mechanisms. They expect vulnerabilities to be patched automatically yet fear the disruptions that

automatic patches could bring. Manufacturers see this market and its demands and choose to invest their time and budget on what will make them the most money. They ship out devices that are “just secure enough” to avoid immediate liability, while remaining vulnerable to the next Mirai.

An Analysis of Budget IoT Devices

The preceding sections have established that budget IoT devices are vulnerable by design. This is due to cost-cutting measures imposed by a cut-throat market. The devices use cheaper materials, outdated components and software, and need to be simplified for consumer usability. To move beyond theoretical risk assessment and document exactly how these devices are left vulnerable, this research next describes the development of a custom network sniffing script. The script was designed to provide a fast, simple way to analyze all devices on a network and find which ones may have vulnerable open ports in a home/local network environment.

The script pursues a three-stage methodology. First, it discovers all active devices on the local subnet using ARP-based scanning. Second, it performs targeted port scans against each discovered device, focusing on ports traditionally associated with IoT vulnerabilities. Telnet (23), HTTP (80), RTSP (554), MQTT (1883), HTTP-alt (8080), and the Tuya smart plug protocol (6668). Third, it optionally captures live network traffic to identify IoT cloud communications and unencrypted protocols. Each device is then classified by type (camera, smart plug, embedded IoT device, or computer), assigned a numerical risk score from 0 to 100, and labeled with a severity level (LOW, MEDIUM, HIGH, or CRITICAL).

Development Environment and Language Selection

The script was developed using Visual Studio Code (VS Code) as the integrated development environment. VS Code was selected for its lightweight architecture, native support of python debugging, and extensive library of extensions that simplified packet capture analysis. The integrated terminal present allowed for real-time execution and testing of network interception routines without switching between applications.

Python 3.9 was chosen as the implantation language due to three factors: its expansive catalog of networking libraries, its rapid prototyping capabilities, and its readability for future analysis of this research. Unlike lower-level languages such as C or Rust, Python abstracts away a lot of the socket-level complexity, allowing the analysis to focus on the network traffic rather than memory management.

Library Selection and Justification

The script was built around two external libraries. The first being Scapy, which is used for packet sniffing, dissection, and reconstruction. Scapy provides the ability to capture live traffic, filter by protocol, and inspect individual packets without writing low-level socket handlers. Scapy enables passive observation of DNS queries and unecrypted protocols such as Telnet and MQTT. The second library is Nmap, accessed through the python-nmap wrapper. While Scapy handles live packet capture and inspection, python-nmap provides access to Nmap's port scanning and service detection capabilities. This allowed the script to perform rapid port scans across all 254 IP addresses in the local subnet without reinventing fundamental networking tools.

The python-nmap library was specifically chosen over manual socket scanning for three reasons. First Nmap's TCP SYN scan (the default -sS mode) is significantly faster than Python's

socket.connect.ex() method, completing a full /24 subnet scan in seconds rather than minutes. Second, Nmap includes service version detection (-sV), which can identify not only that port 80 is open, but that it is running a specific HTTP server implementation. Third, python-nmap handles concurrency internally, eliminating the need for the script to manage threading or asynchronous I/O. The integration of these two libraries created a complementary workflow. Python-nmap performed the initial reconnaissance, identifying which devices on the network had open IoT relevant ports. Once vulnerable devices were located, Scapy took over to perform packet inspection.

Risk Scoring Methodology

A critical component in this script is the implementation of a risk scoring engine that assigns each device a vulnerability rating. Each open port contributes a weighted score based on its historical association with IoT exploits. Telnet (port 23) contributes 40 points due to its transmission of plaintext credentials and history with the Mirai botnet. RTSP (port 554) contributes 25 points because camera streams often lack authentication. MQTT (port 1883) contributes 20 points as IoT messaging frequently remains unencrypted. HTTP ports 80 and 8080 contribute 10 points each, as exposed websites may leak configuration data or accept default credentials. An additional 2 points are added for each open port beyond the first, reflecting the principle that an increased attack surface results with increased risk. The final score is capped at 100, and devices are labeled as CRITICAL (70+), HIGH (40-69), MEDIUM (20-39), and LOW (0-19).

Device Classification

The script includes a device categorizer that classifies each host based on its open ports. Cameras are identified by the presence of RSTP (port 554). Smart plugs are flagged by Tuya protocol port 6668 or alternative HTTP port 8080. MQTT devices indicate IoT messaging hardware, Computers are distinguished by SSH (port 22) or SMB (port 445). Devices with open HTTP port 80 with few additional ports are classified as embedded IoT devices. This classification enables the report to present finding in consumer relevant terms rather than raw port numbers.

Script Prototyping

The script was prototyped and tested on a MacBook Air equipped with an Apple M1 processor and 8GB of RAM running MacOS Tahoe 26.4. This hardware selection was deliberate: if a consumer, security researcher, or malicious actor were to execute this code in a real-world scenario, it would most likely be run from a laptop inside someone's home, a coffee shop, a library, or another open location. Unlike a desktop workstation or server rack, a laptop is portable, discreet, and capable of operating on battery power. The MacBook Air's lightweight form factor and silent operation further mirror the kind of hardware a malicious actor might deploy to remain undetected while conducting reconnaissance on a target network. The 8GB RAM ceiling proved sufficient for capturing packets.

Prior to writing any executable code, the script's logic was captured in pseudocode. This step proved essential for three reasons. First, the pseudocode allowed the algorithm to be refined without distraction of Python or library-specific syntax. Second, the pseudocode served as documentation for the script's intended behavior before a single line of Python was written, enabling early identification of logical gaps. Lastly, the pseudocode provides a non-language specific record of the methodology used to create the script, ensuring that the approach remains

understandable to readers regardless of their familiarity with Python or the specific libraries employed.

Psuedocode for the Vulnerability Scanner

The complete pseudocode developed prior to implementation is presented below.

```
Require: network_range (auto-detected if not provided)
Ensure: security report with risk scores and severity labels

// STAGE 1: DISCOVER DEVICES

devices ← empty list
PERFORM Nmap ARP scan (-sn -PR) on network_range
for each active host do
    devices ← devices + {ip, hostname}
    PRINT "Found: " + ip
end for

// STAGE 2: SCAN PORTS & ASSESS RISK

DEFINE IOT_PORTS = [23, 80, 443, 554, 1883, 8080, 6668]
vulnerable ← empty list
for each device in devices do
    PERFORM Nmap SYN scan on IOT_PORTS
    open_ports ← list of open ports
  if open_ports not empty then
        // Classify device
    if 554 IN open_ports then
      type ← "camera"
    else if 6668 IN open_ports OR 8080 IN open_ports then
      type ← "smart_plug"
    else
      type ← "iot_device"
    end if

        // Calculate risk score (0-100)
        score ← 0
    if 23 IN open_ports then
      score ← score + 40
    end if
    if 554 IN open_ports then
      score ← score + 25
    end if
    if 1883 IN open_ports then
      score ← score + 20
    end if
    if 80 OR 8080 IN open_ports then
      score ← score + 10
    end if
        score ← score + (len(open_ports) * 2)
    if score > 100 then
      score ← 100
    end if

        // Assign severity label
    if score ≥ 70 then
      severity ← "CRITICAL"
    else if score ≥ 40 then
      severity ← "HIGH"
    else if score ≥ 20 then
      severity ← "MEDIUM"
```

```
    else
      severity ← "LOW"
    end if

        vulnerable ← vulnerable + {ip, type, score, severity}
        PRINT ip + " → " + type + " — " + score + "/100"
  end if
end for

// STAGE 3: OPTIONAL PACKET CAPTURE

events ← empty list
packet_count ← 0
if packet capture enabled then
    START sniffing for user-specified duration
  for each packet do
        packet_count ← packet_count + 1
    if DNS query contains "tuya" OR "amazon" OR "hikvision" then
            events ← events + "IoT cloud DNS call"
    end if
    if TCP port 1883 OR 23 is detected then
            events ← events + "Unencrypted protocol"
    end if
  end for
end if

// GENERATE REPORT

report ← "=== IoT SECURITY ANALYSIS REPORT ==="
report ← report + "High Risk Devices: " + len(vulnerable)
for each device in vulnerable do
    report ← report + device.ip + " (" + device.type + ")"
    report ← report + "Risk: " + device.score + " (" + device.severity + ")"
end for
if packet_count > 0 then
    report ← report + "Packets: " + packet_count
    report ← report + "Events: " + len(events)
end if
SAVE report to timestamped file
PRINT report
```

Figure 2 – Psuedocode for Scanning Script

Script Compilation and Execution

Following the logic outlined in Figure 2, the script was implemented in Python. Prior to execution, the required dependencies must be installed: python-nmap for port scanning and Scapy for optional packet capture. The system must also have nmap installed, as python-nmap is simply a wrapper for python usability.

The script was tested on macOS Tahoe 26.4 running on a MacBook Air M1 with 8GB of RAM. Execution requires sudo or root privileges as packet capture and ARP scanning needs raw socket access. No compilation is required, as Python is an interpreted language. The script is executed with the sudo python3 iot_scanner.py command in the macOS terminal. Upon

execution, the user is prompted for a network range. Pressing Enter attempts to auto-detect the local subnet. The script then performs Stage 1 and Stage 2 automatically, displaying discovered devices in real time with the risk scores being displayed in the final report. The user is then prompted whether they would like to run Stage 3 and, if confirmed, the user specifies the packet capture duration. The script then saves a timestamped report to the current working directory. The network sniffing script is publicly available on GitHub (Arellano, 2026).

Test Device Selection

To evaluate the network vulnerabilities of budget IoT devices in a real-world scenario, two devices were purchased from Amazon. Both devices were selected based on their low-price points, consumer popularity (indicated by "Amazon's choice" designations), and their representation of common IoT categories in an indoor security camera and a smart plug.

The first test device was a GNCC brand indoor security camera, priced at $12.99. This device represents the low-end category of the home surveillance market. From a security perspective, several of the device's specifications warrant investigation. The device requires the "OSAIO" mobile application for setup and control. This is a third-party app likely developed by a different manufacturer rather than GNCC itself. This is a common cost cutting practice where brands rebrand generic hardware and software. The camera supports both optional cloud storage, and local SD card storage, suggesting the device communicates with a cloud-based server. The listing also states that the device only supports 2.4GHz Wi-Fi, not 5GHz, which is a common limitation in budget IoT devices due to the lower price of 2.4GHz-only radio chips. The camera transmits video and event data across the network, making it an ideal candidate for the port scanning and traffic analysis script created.

The second test device was a Ghome-branded smart plug, priced at $8.99. This device represents the most affordable tier of smart home IoT devices. Several specifications from the device listing raised security concerns. The plug supports connection via the Ghome, Smart Life, or Tuya app. This is significant due to the Tuya platform running outdated chipsets that have documented vulnerabilities. The device requires a 2.4GHz Wi-Fi connection only, again reflecting the cost-cutting techniques used by manufacturers. Importantly, the listing notes that to use Alexa or Google Home, the product "cannot be connected directly" and must first be connected to the Ghome app, implying a cloud-dependent architecture where all voice commands are relayed through manufacturer servers. The smart plug requires persistent outbound connections due to its cloud-based infrastructure. This behavior makes it suitable for observing IoT cloud communication patterns and testing for exposed services on ports associated with Tuya (port 6668).

Network Discovery Result

The network scan revealed a concerning pattern of exposed services across budget IoT devices on the test network. A summary of the findings by device type is presented in Table 1.

Table 1: IoT Security Scan Results with Vulnerability Mapping

IP Address	Device Type	Risk Score	Severity	Vulnerability
192.168.0.1	embedded_iot	14	LOW	HTTP admin interface exposed[1]
192.168.0.136	smart_plug	2	LOW	Tuya protocol port open[2]
192.168.0.143	smart_plug	24	MEDIUM	HTTP admin interface exposed[1]
192.168.0.163	embedded_iot	14	LOW	HTTP admin interface exposed[1]
192.168.0.171	camera	27	MEDIUM	RTSP camera stream exposure[3]
192.168.0.205	smart_plug	24	MEDIUM	HTTP admin interface exposed[1]

There script scanned 11 devices on the network, but we will perform a detailed discussion on five of the six devices shown in Table 1.

The device classified as a camera (192.168.0.171) had port 554 (RTSP) open without apparent authentication requirements. RTSP is the Real Time Streaming Protocol commonly used by IP cameras to deliver video feeds. When left unsecured, an attacker on the same local network can connect to this port and view live video camera streams using any standard media player. This represents a direct privacy violation. As documented in the earlier section on IP camera attacks, malicious actors have historically scraped unsecured RTSP feeds to create publicly accessible websites of private home video streams. The camera's risk score of 27 (MEDIUM) reflects this vulnerability.

There were two previously purchased VOCOlink brand smart plugs (192.168.0.143 and 192.168.0.205) on the network that had not been purchased for the research that had ports 80 and 8080 open. The presence of HTTP interfaces on smart home devices is concerning for two reasons. First, these web interfaces often accept default credentials that are documented on public databases. Second, even when credentials are changed, HTTP transmits login information in plaintext, allowing any attacker with network visibility to capture usernames and passwords. The risk score of 24 (MEDIUM) is primarily caused by the presence of port 80, though the score does not illustrate the full risk of credential interception during live sessions.

The smart plug originally purchased for the research (192.168.0.136) has port 6668 open. This port is associated with the Tuya smart home platform. This port is used for device provisioning and cloud communication. While the script assigned a LOW risk score of 2 to this

device, the presence of this port indicates that the device uses the BK7231 chipset. As documented in the Tuya CloudCutter github repo (Nassar et al., n.d.), devices with this chipset are vulnerable to permanent disconnection from the cloud and local control takeover.

Lastly, there is a network gateway (192.168.0.1) that had HTTP and HTTPS ports 80 and 443 open. While common for administrative access, this exposes the router's web interface to any device on the local network. If default credentials remain unchanged an attacker could gain administrative control over the entire network. The LOW risk score of 14 reflects that HTTP alone is less dangerous than Telnet or RTSP, but the risk escalates significantly when combined with default credential usage.

Risk Score Distribution

Of the 11 devices discovered, three devices received risk scores in the MEDIUM range (20-39) and 2 devices scored in the LOW range (0-19). No devices receive HIGH or CRITICAL scores on the test network. The average risk score among IoT-classified devices (cameras and smart plugs) was 20.8, compared to 14 for the network gateway and 2 for Tuya device. While no devices in this test reached the CRITICAL threshold, the presence of MEDIUM risk devices with RTSP and HTTP exposures confirms that budget IoT devices continue to ship with vulnerable configurations. The absence of Telnet (port 23) on any discoverable device suggests that manufacturers have responded to the Mirai botnet by disabling the most egregious vulnerable service, though significant risks remain.

Threat Analysis

The vulnerabilities documented in this scan could be exploited by a malicious actor with local network access. Several realistic attack scenarios emerge from the findings.

Scenario 1: Unauthorized Camera Surveillance

An attacker who gains access to the local network can discover the camera at 192.168.0.171 using a simple port scan. Once identified as hosting RTSP on port 554, the attacker can connect using an RTSP client. If the stream requires no authentication, the attacker gains live video access to the camera's field of view. In a home environment this could include bedrooms, living areas, or security camera feeds intended for private use only. The attacker could also use the camera as a pivot point to scan for other vulnerable devices on the network.

Scenario 2: Smart Plug Control and Network Pivot

The VOCOlinc smart plugs at 192.168.0.143 and 192.168.0.205 expose HTTP interfaces on ports 80 and 8080. An attacker can access these web interfaces and attempt default credentials. If successful, the attacker gains the ability to toggle the connected appliance on and off. While seemingly low impact, this capability can be weaponized, just as similar IoT devices were once used in the Mirai attacks. Additionally, the smart plug's network connection provides a foothold from which the attacker can scan internal network segments that may be inaccessible from the attacker's original position.

Scenario 3: Gateway Compromise

The network gateway at 192.168.0.1 exposes its web interface on ports 80 and 443. If the consumer has not changed the default administrative passwords, which is a common behavior documented in the consumer frustration section, an attacker can log in and modify router settings. This could include redirecting DNS queries to malicious servers, disabling firewall rules, or capturing all traffic passing through the gateway. Gateway compromise represents a worst-case scenario because the router controls all network traffic for every device in the home.

Scenario 4: Tuya CloudCutter Permanent Device Takeover

The Ghome smart plug at 192.168.0.136 communicates over Tuya's cloud infrastructure, as indicated by open port 6668. This port suggests that the device uses a BK7231 chipset. This is a low-cost component that prioritizes affordability over security. An attacker with local network access can execute the Tuya CloudCutter exploit against this device. This attack forces the smart plug into a device provisioning mode, then executes a man-in-the-middle attack during the provisioning process, intercepting and replacing the device's cloud authentication keys. Once the exploit succeeds, the device is permanently disconnected from Tuya's cloud servers, and the owner can no longer control the plug.

Comparison to Consumer Expectation

The findings from this scan highlight the gap between consumer perception and reality. The camera owner likely purchased the device for security and peace of mind, unaware that it exposes an unauthenticated video stream to anyone on the network. The smart plug owner is unaware that the device's HTTP interface could allow an attacker to take complete control over his outlet. The gateway owner may have connected the router provided by their internet service provider, unaware that its exposed web interface is a potential attack surface.

These devices function exactly as designed; they are not defective or malfunctioning. The vulnerabilities identified are intentional design choices made by manufacturers to reduce costs, simplify setup, or accelerating product development. The consumer, lacking the technical ability to discover these vulnerabilities, continues to use the devices in blissful ignorance.

Conclusion

The rise of budget-friendly IoT devices has made access to smart home technology attainable for the average consumer. However, as this research has demonstrated, the low-price tags of these devices hide a significant and invisible cost. The cost cutting strategies employed by manufacturers, like reusing deprecated software libraries, selecting hardware components without security features, abandoning firmware updates shortly after product release, and prioritizing usability over security, have transformed home networks into massive attack surfaces.

The script developed for this research, an IoT security analyzer, successfully discovered 11 active devices on a local subnet and identified six devices with exploitable configurations. Among the findings, a budget camera exposed a potentially unauthenticated video stream, multiple smart plugs presented open HTTP interfaces susceptible to default credential attacks, and a Tuya-based smart plug revealed port signatures associated with a known exploit via Tuya CloudCutter. These findings are the predictable outcome of an industry that competes on price while treating security as an afterthought.

An attacker with even temporary local network access could view private camera feeds, toggle connected appliances, compromise a network gateway, or permanently hijack a smart plug. Each of these outcomes violates consumer expectations of privacy, safety and control.

Several recommendations emerge from this analysis. For consumers, the most immediate defenses are network segmentation, changing all default credentials during device setup, and regularly checking for firmware updates. For manufacturers, balance must be found when choosing to prioritize material cost against security. Manufacturers must include unique

credentials for each device, disabling unnecessary ports by default, and building the infrastructure to provide security updates. For regulators and standards bodies, the research shows that the market will not correct these failures, as consumers cannot evaluate security at the point of purchase, and manufacturers face no meaningful penalty for distributing vulnerable devices.

The central thesis of this paper has been supported by both the literature review and the empirical findings. The research shows that budget IoT devices introduce exploitable network vulnerabilities as a direct consequence of manufacturer cost-cutting strategies. The Mirai botnet demonstrated the risk of default credentials. The CloudCutter exploit reveals the danger of outdated chipset selection. The camera and smart plug findings in this paper confirm that these vulnerabilities persist in devices sold today. If these dynamics between manufacturers and cost-cutting strategies stay the same, budget IoT devices will remain an ongoing threat to home network security for the foreseeable future.

References

Antonakakis, M., April, T., Bailey, M., Bernhard, M., Bursztein, E., Cochran, J., Durumeric, Z., Halderman, J. A., Invernizzi, L., Kallitsis, M., Kumar, D., Lever, C., Ma, Z., Mason, J., Menscher, D., Seaman, C., Sullivan, N., Thomas, K., & Zhou, Y. (2017). Understanding the Mirai botnet. In Proceedings of the 26th USENIX Security Symposium (SEC '17) (pp. 1093–1110). USENIX Association. https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis

Arellano, J. (2026). Juliogerarellano/CYBV-498-Budget-IoT-Devices-and-their-vulnerabilities: Repository of final cyber operations capstone. GitHub. https://github.com/juliogerarellano/CYBV-498-Budget-IoT-Devices-and-Their-Vulnerabilities

Blinder, C. (2024, April 23). Average number of smart devices in a home 2026. ConsumerAffairs. https://www.consumeraffairs.com/homeowners/average-number-of-smart-devices-in-a-home.html

Chen, Y., Ma, F., Zhang, Y., He, Y., Wang, H., & Li, Q. (2024). AutoFirm: Automatically Identifying Reused Libraries inside IoT Firmware at Large-Scale. https://doi.org/10.48550/arxiv.2406.12947

iQor. (2018, January 30). The Smart Home is creating frustrated consumers: More than 1 in 3 US adults experience issues setting up or operating a connected device. The Smart Home is Creating Frustrated Consumers: More than 1 in 3 US Adults Experience Issues Setting up or Operating a Connected Device | Business Wire. https://web.archive.org/web/20210307072006/https:/www.businesswire.com/news/home/

20180130005463/en/The-Smart-Home-is-Creating-Frustrated-Consumers-More-than-1-in-3-US-Adults-Experience-Issues-Setting-up-or-Operating-a-Connected-Device

Jaigirdar, F. T., Jayatilaka, A., & Babar, M. A. (2026). Software vulnerability management in IoT systems: a systematic mapping study. Cybersecurity, 9(1), Article 96. https://doi.org/10.1186/s42400-025-00543-6

Miettinen, M., Sadeghi, A.-R., Marchal, S., Asokan, N., Hafeez, I., & Tarkoma, S. (2017). IoT SENTINEL: Automated Device-Type Identification for Security Enforcement in IoT. Proceedings of the International Conference on Distributed Computing Systems, 2177–2184. https://doi.org/10.1109/ICDCS.2017.283

Murat, K., Topyła, D., Zdulski, K., Marzęcki, M., Bieniasz, J., Paczesny, D., & Szczypiorski, K. (2024). Security Analysis of Low-Budget IoT Smart Home Appliances Embedded Software and Connectivity. Electronics (Basel), 13(12), 2371. https://doi.org/10.3390/electronics13122371

Nassar, K., Clement, T., & Groenendijk, J. (n.d.). Tuya-cloudcutter/tuya-cloudcutter: A tool that disconnects tuya IOT devices from the cloud, allowing them to run completely locally. GitHub. https://github.com/tuya-cloudcutter/tuya-cloudcutter

Rawat, R. S., Diwakar, M., & Srivastava, P. (2025). Investigation of IoT Botnets Behaviour and Analysis. 2025 International Conference on Networks and Cryptology (NETCRYPT), 1683–1688. https://doi.org/10.1109/NETCRYPT65877.2025.11102265

Ronen, E., O'Flynn, C., Shamir, A., & Weingarten, A.-O. (2017). IoT Goes Nuclear: Creating a ZigBee Chain Reaction. Proceedings - IEEE Symposium on Security and Privacy, 195–212. https://doi.org/10.1109/SP.2017.14

Siryi, O. (2025). Challenges with device ota updates and their solutions. SoftServe. https://www.softserveinc.com/en-us/blog/challenges-with-device-ota-updates

Teicher, J. (2025, November 17). *The history of the first IOT device*. IBM. https://www.ibm.com/think/topics/iot-first-device

Williams, S. (2025, November 28). One in three Britons struggle to connect Smart Tech at home. IT Brief UK. https://itbrief.co.uk/story/one-in-three-britons-struggle-to-connect-smart-tech-at-home

Wolfe, E., & Ries, B. (2019, December 13). A hacker accessed a family's ring security camera and told their 8-year-old daughter he was santa claus | CNN business. CNN. https://www.cnn.com/2019/12/12/tech/ring-security-camera-hacker-harassed-girl-trnd